

knn_naivebayes

2026-05-01

Performing K-Nearest Neighbors vs. Naive Bayes on Binary and Multiclass datasets.

##Naive Bayes data_processing.R ran in this code to ensure .Rmd file can run smoothly.

```
#Loading training set data
binary_train <- read.csv("~/Downloads/binary_train.csv")
multiclass_train <- read.csv("~/Downloads/multiclass_train.csv")
binary_test <- read.csv("~/Downloads/binary_test.csv")
multiclass_test <- read.csv("~/Downloads/multiclass_test.csv")

#Initially run the data_processing.R code to prep for scaling datasets
library(caret)
```

Loading required package: ggplot2

Loading required package: lattice

```
# Identify the feature type from the data setidentify_feature_types

identify_feature_types <- function(data) {

  numeric_cols <- names(data)[sapply(data, is.numeric)]

  binary_cols <- c()
  continuous_cols <- c()

  for (col in numeric_cols) {
    unique_vals <- unique(data[[col]])
    unique_vals <- unique_vals[!is.na(unique_vals)]

    if (length(unique_vals) == 2) {
      binary_cols <- c(binary_cols, col)
    } else {
      continuous_cols <- c(continuous_cols, col)
    }
  }

  return(list(
    binary = binary_cols,
    continuous = continuous_cols
  ))
}
```

```

# -----
# Fit scaler for Training data
# -----
fit_scaler <- function(train_data, continuous_cols) {

  scaler <- preProcess(
    train_data[, continuous_cols],
    method = c("center", "scale")
  )

  return(scaler)
}

# -----
# Transformation the data
# -----
transform_data <- function(data, scaler, continuous_cols) {

  data_scaled <- data

  data_scaled[, continuous_cols] <- predict(
    scaler,
    data[, continuous_cols]
  )

  return(data_scaled)
}

# -----
# Data pipeline
# -----
prepare_data <- function(train_data, test_data) {

  # Step 1: Identify feature types
  feature_types <- identify_feature_types(train_data)

  feature_types$binary
  feature_types$continuous
  cat("Binary Features    : ",feature_types$binary, "\n")
  cat("Continuous Features: ",feature_types$continuous, "\n")

  continuous_cols <- feature_types$continuous

  # Step 2: Fit scaler ONLY on training
  scaler <- fit_scaler(train_data, continuous_cols)

  # Step 3: Transform both datasets
  train_processed <- transform_data(train_data, scaler, continuous_cols)
  test_processed  <- transform_data(test_data, scaler, continuous_cols)

  return(list(
    train = train_processed,
    test  = test_processed,
  ))
}

```

```

    scaler = scaler,
    feature_types = feature_types
  ))
}

#Onto scaling

#Scaling binary dataset
binary_result <- prepare_data(binary_train, binary_test)

## Binary Features      : Diabetes_binary HighBP HighChol CholCheck Smoker Stroke HeartDiseaseorAttack P
## Continuous Features: BMI GenHlth MentHlth PhysHlth Age Education Income

binary_train_scaled <- binary_result$train
binary_test_scaled <- binary_result$test

#Scaling multiclass set
multiclass_result <- prepare_data(multiclass_train, multiclass_test)

## Binary Features      : HighBP HighChol CholCheck Smoker Stroke HeartDiseaseorAttack PhysActivity Frui
## Continuous Features: Diabetes_012 BMI GenHlth MentHlth PhysHlth Age Education Income

multiclass_train_scaled <- multiclass_result$train
multiclass_test_scaled <- multiclass_result$test

binary_target <- "Diabetes_binary"
multiclass_target <- "Diabetes_012"

# Convert targets to factors
binary_train_scaled[[binary_target]] <- as.factor(binary_train_scaled[[binary_target]])
binary_test_scaled[[binary_target]] <- as.factor(binary_test_scaled[[binary_target]])

multiclass_train_scaled[[multiclass_target]] <- as.factor(multiclass_train_scaled[[multiclass_target]])
multiclass_test_scaled[[multiclass_target]] <- as.factor(multiclass_test_scaled[[multiclass_target]])

#Applying SMOTE to handle class imbalances
library(ROSE)

## Loaded ROSE 0.0-4

set.seed(12)
binary_train_balanced <- ROSE(Diabetes_binary ~ ., data = binary_train_scaled, p = 0.5, seed = 123)$data

#Performing Gaussian Naive Bayes on Binary
library(e1071)

##
## Attaching package: 'e1071'

## The following object is masked from 'package:ggplot2':
##
## element

```

```

set.seed(12)
nb_binary <- naiveBayes(Diabetes_binary ~ ., data = binary_train_balanced)
nb_binary.pred <- predict(nb_binary, binary_test_scaled)
nb_binary.prob <- predict(nb_binary, binary_test_scaled, type = "raw")

#Evaluate
library(caret)
cm_binary <- confusionMatrix(nb_binary.pred, binary_test_scaled$Diabetes_binary, positive = "1")
print(cm_binary)

```

```

## Confusion Matrix and Statistics
##
##              Reference
## Prediction      0      1
##              0 32891  2351
##              1 10776  4719
##
##              Accuracy : 0.7413
##              95% CI : (0.7374, 0.7451)
##              No Information Rate : 0.8607
##              P-Value [Acc > NIR] : 1
##
##              Kappa : 0.2806
##
##              Mcnemar's Test P-Value : <2e-16
##
##              Sensitivity : 0.66747
##              Specificity : 0.75322
##              Pos Pred Value : 0.30455
##              Neg Pred Value : 0.93329
##              Prevalence : 0.13935
##              Detection Rate : 0.09301
##              Detection Prevalence : 0.30540
##              Balanced Accuracy : 0.71035
##
##              'Positive' Class : 1
##

```

#Get resulting confusion matrix and statistics analysis

```

# Store metrics
nb_binary_metrics <- data.frame(
  Model = "Naive Bayes (Balanced)",
  Accuracy = cm_binary$overall["Accuracy"],
  Precision = cm_binary$byClass["Precision"],
  Recall = cm_binary$byClass["Sensitivity"],
  F1 = cm_binary$byClass["F1"],
  Specificity = cm_binary$byClass["Specificity"]
)

```

#Naive Bayes Gaussian on Multiclass Dataset

```

nb_multiclass.fit <- naiveBayes(
  Diabetes_012 ~ ., # Target is 0,1,2

```

```

data = multiclass_train_scaled
)

nb_multiclass.pred <- predict(nb_multiclass.fit, multiclass_test_scaled)
nb_multiclass.prob <- predict(nb_multiclass.fit, multiclass_test_scaled, type = "raw")

#Evaluate
cm_multiclass <- confusionMatrix(nb_multiclass.pred, multiclass_test_scaled$Diabetes_012)
print(cm_multiclass)

## Confusion Matrix and Statistics
##
##               Reference
## Prediction  -0.425204906276596  1.00719984273144  2.43960459173948
##   -0.425204906276596          34368           538           3033
##   1.00719984273144           373           23           104
##   2.43960459173948          7987          377           3934
##
## Overall Statistics
##
##               Accuracy : 0.7554
##               95% CI : (0.7516, 0.7591)
##   No Information Rate : 0.8421
##   P-Value [Acc > NIR] : 1
##
##               Kappa : 0.2726
##
##   McNemar's Test P-Value : <2e-16
##
## Statistics by Class:
##
##               Class: -0.425204906276596 Class: 1.00719984273144
## Sensitivity                0.8043                0.0245203
## Specificity                0.5541                0.9904215
## Pos Pred Value             0.9059                0.0460000
## Neg Pred Value             0.3468                0.9817863
## Prevalence                 0.8421                0.0184875
## Detection Rate             0.6774                0.0004533
## Detection Prevalence       0.7478                0.0098547
## Balanced Accuracy          0.6792                0.5074709
##
##               Class: 2.43960459173948
## Sensitivity                0.55636
## Specificity                0.80846
## Pos Pred Value            0.31989
## Neg Pred Value            0.91839
## Prevalence                 0.13937
## Detection Rate            0.07754
## Detection Prevalence      0.24239
## Balanced Accuracy          0.68241

# Calculate macro F1
nb_macro_f1 <- mean(cm_multiclass$byClass[, "F1"], na.rm = TRUE)

```

```
nb_macro_f1 <- mean(cm_multiclass$byClass[, "F1"], na.rm = TRUE)
```

```
#Using numeric indices
```

```
nb_multiclass_metrics <- data.frame(
  Model = "Naive Bayes",
  Accuracy = cm_multiclass$overall["Accuracy"],
  Macro_F1 = nb_macro_f1,
  Class0_F1 = cm_multiclass$byClass[1, "F1"], # First class
  Class1_F1 = cm_multiclass$byClass[2, "F1"], # Second class
  Class2_F1 = cm_multiclass$byClass[3, "F1"] # Third class
)
```

```
print(nb_multiclass_metrics)
```

```
##           Model Accuracy Macro_F1 Class0_F1 Class1_F1 Class2_F1
## Accuracy Naive Bayes 0.7553659 0.4301002 0.8520957 0.03198887 0.4062161
```

```
library(class)
```

```
set.seed(12)
```

```
#K-NN on Balanced Binary Dataset
```

```
train.X.balanced <- binary_train_balanced[, !names(binary_train_balanced) %in% "Diabetes_binary"]
```

```
train.Direction.balanced <- binary_train_balanced$Diabetes_binary
```

```
test.X <- binary_test_scaled[, !names(binary_test_scaled) %in% "Diabetes_binary"]
```

```
test.Y <- binary_test_scaled$Diabetes_binary
```

```
#Testing K = 3, 5, 7, 11, 15
```

```
k_values <- c(3, 5, 7, 11, 15)
```

```
balanced_results <- data.frame()
```

```
for(k in k_values) {
```

```
  set.seed(1)
```

```
  knn.pred <- knn(train.X.balanced, test.X, train.Direction.balanced, k = k)
```

```
  cm <- confusionMatrix(knn.pred, test.Y, positive = "1")
```

```
  balanced_results <- rbind(balanced_results, data.frame(
```

```
    K = k,
```

```
    Accuracy = cm$overall["Accuracy"],
```

```
    Precision = cm$byClass["Precision"],
```

```
    Recall = cm$byClass["Sensitivity"],
```

```
    F1 = cm$byClass["F1"],
```

```
    Specificity = cm$byClass["Specificity"]
```

```
  ))
```

```
}
```

```
print(balanced_results)
```

```
##           K Accuracy Precision Recall F1 Specificity
## Accuracy  3 0.7674675 0.3069889 0.5318246 0.3892743 0.8056198
## Accuracy1 5 0.7778544 0.3245050 0.5493635 0.4080046 0.8148487
## Accuracy2 7 0.7828606 0.3340341 0.5618105 0.4189652 0.8186502
## Accuracy3 11 0.7870982 0.3430878 0.5770863 0.4303344 0.8211006
## Accuracy4 15 0.7873544 0.3434633 0.5770863 0.4306296 0.8213983
```

```
best_k_balanced <- balanced_results[which.max(balanced_results$F1), "K"]
best_f1_balanced <- max(balanced_results$F1)
```

```
cat("\n Best K for BALANCED data:", best_k_balanced,
    "with F1 =", round(best_f1_balanced, 4), "\n")
```

```
##
## Best K for BALANCED data: 15 with F1 = 0.4306
```

```
# Store best KNN binary results
knn_binary_metrics <- balanced_results[balanced_results$K == best_k_balanced, ]
knn_binary_metrics$Model <- paste("KNN (K=", best_k_balanced, ")", sep="")
```

```
##K Nearest Neighbors
```

```
library(class)
#Multiclass KNN Balanced
train.X <- multiclass_train_scaled[, !names(multiclass_train_scaled) %in% "Diabetes_012"]
train.Y <- multiclass_train_scaled$Diabetes_012
test.X <- multiclass_test_scaled[, !names(multiclass_test_scaled) %in% "Diabetes_012"]
test.Y <- multiclass_test_scaled$Diabetes_012

#Testing K = 3, 5, 7, 11, 15
multiclass_results <- data.frame()

for(k in k_values) {
  set.seed(1)
  knn.pred <- knn(train.X, test.X, train.Y, k = k)
  cm <- confusionMatrix(knn.pred, test.Y)

  # Calculate macro F1 (average across classes)
  macro_f1 <- mean(cm$byClass[, "F1"], na.rm = TRUE)
  multiclass_results <- rbind(multiclass_results, data.frame(
    K = k,
    Accuracy = cm$overall["Accuracy"],
    Macro_F1 = macro_f1,
    Class0_F1 = cm$byClass[1, "F1"], # First class
    Class1_F1 = cm$byClass[2, "F1"], # Second class
    Class2_F1 = cm$byClass[3, "F1"] # Third class
  ))
}

print(multiclass_results)
```

```
##           K Accuracy Macro_F1 Class0_F1 Class1_F1 Class2_F1
## Accuracy  3 0.8181800 0.4070936 0.8987502 0.019464720 0.3030659
## Accuracy1  5 0.8300057 0.4018442 0.9054219 0.003968254 0.2961424
## Accuracy2  7 0.8347951 0.5962014 0.9083915          NA 0.2840114
## Accuracy3 11 0.8413978 0.5940916 0.9123834          NA 0.2757998
## Accuracy4 15 0.8431322 0.5873530 0.9134026          NA 0.2613034
```

```

# Best K based on macro F1
best_k_multiclass <- multiclass_results[which.max(multiclass_results$Macro_F1), "K"]
best_macro_f1 <- max(multiclass_results$Macro_F1)

cat("\n Best K for MULTICLASS:", best_k_multiclass,
    "with Macro F1 =", round(best_macro_f1, 4), "\n")

```

```

##
## Best K for MULTICLASS: 7 with Macro F1 = 0.5962

```

```

# Store best KNN multiclass results
knn_multiclass_metrics <- multiclass_results[multiclass_results$K == best_k_multiclass, ]
knn_multiclass_metrics$Model <- paste("KNN (K=", best_k_multiclass, ")", sep="")

```

##KNN Sensitivity Binary and Multiclass Results using ggplot2

```

#KNN Sensitivity Binary Results
library(ggplot2)

binaryKNN_plot <- ggplot(balanced_results, aes(x = K, y = F1)) +
  geom_line(color = "steelblue", size = 1.5) +
  geom_point(color = "steelblue", size = 4) +
  labs(title = "KNN Sensitivity Analysis - Binary Classification",
       x = "Number of Neighbors (K)",
       y = "F1 Score") +
  theme_minimal()

```

```

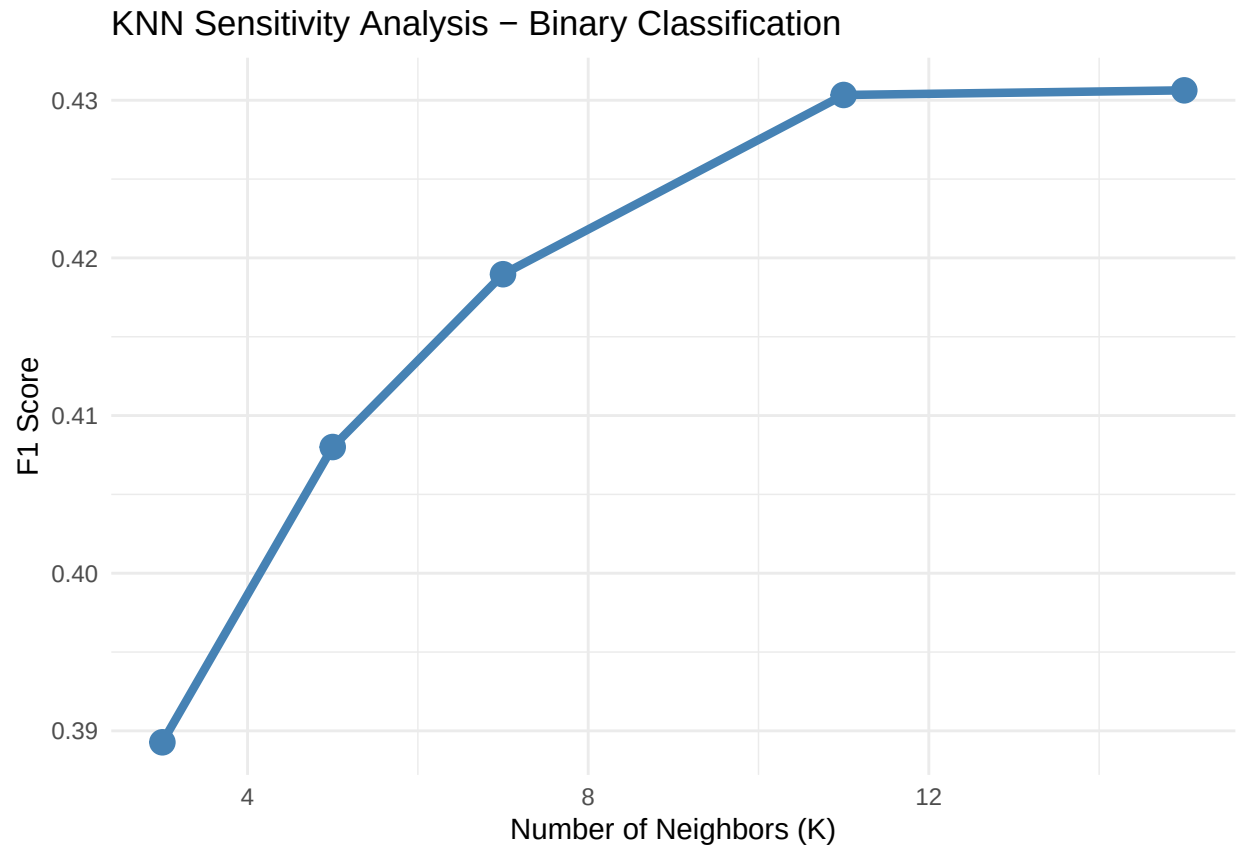
## Warning: Using 'size' aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use 'linewidth' instead.
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.

```

```

print(binaryKNN_plot)

```

```
#KNN Sensitivity Multiclass Results
library(ggplot2)

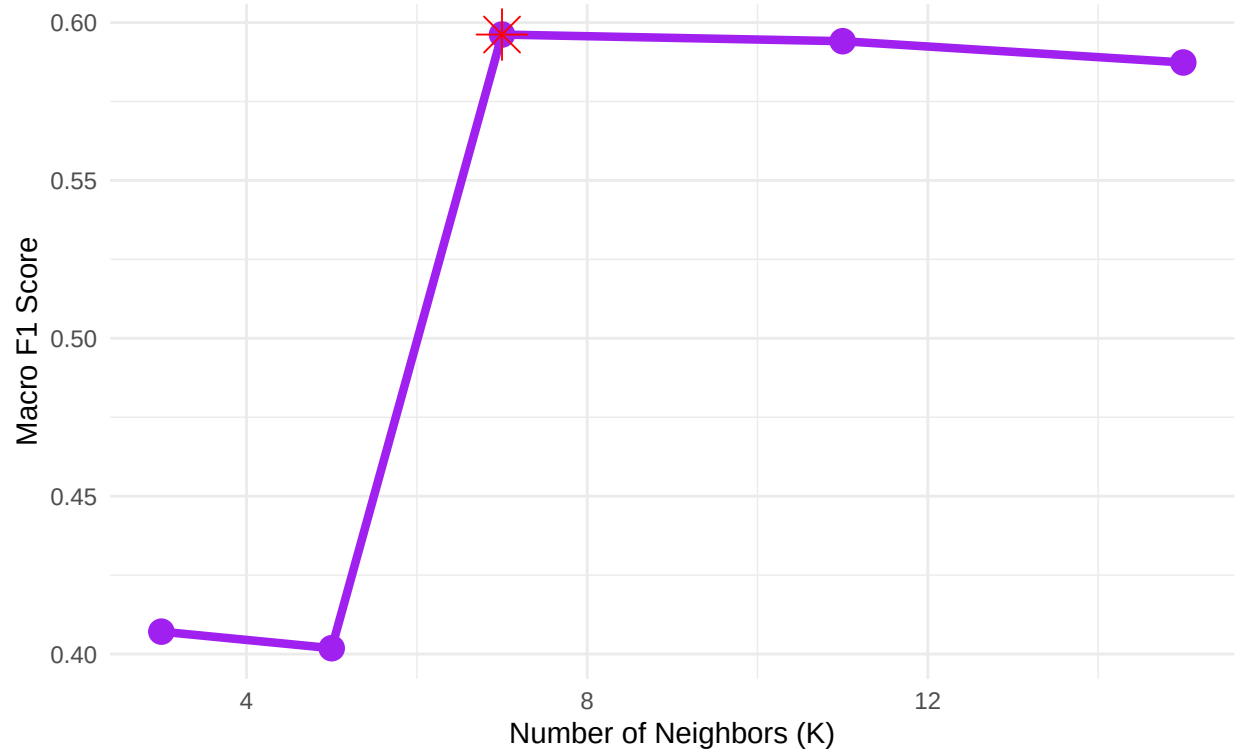
best_k_multi <- multiclass_results$K[which.max(multiclass_results$Macro_F1)]
best_macro_f1 <- max(multiclass_results$Macro_F1)

multiclass_plot <- ggplot(multiclass_results, aes(x = K, y = Macro_F1)) +
  geom_line(color = "purple", size = 1.5) +
  geom_point(color = "purple", size = 4) +
  geom_point(data = multiclass_results[multiclass_results$K == best_k_multi,],
    aes(x = K, y = Macro_F1),
    color = "red", size = 6, shape = 8) +
  labs(title = "KNN Sensitivity Analysis - Multiclass Classification",
    subtitle = paste("Best K =", best_k_multi, "| Macro F1 =", round(best_macro_f1, 4)),
    x = "Number of Neighbors (K)",
    y = "Macro F1 Score") +
  theme_minimal()

print(multiclass_plot)
```

KNN Sensitivity Analysis – Multiclass Classification

Best K = 7 | Macro F1 = 0.5962



```
if(nb_binary_metrics$F1 > knn_binary_metrics$F1) {  
  binary_winner <- "Naive Bayes"  
} else {  
  binary_winner <- "KNN"  
}
```

```
#Binary NB vs KNN Comparison  
cat("\n BINARY CLASSIFICATION:\n")
```

```
##  
## BINARY CLASSIFICATION:
```

```
cat("\nNaive Bayes (Balanced):")
```

```
##  
## Naive Bayes (Balanced):
```

```
cat("\n Accuracy:", round(nb_binary_metrics$Accuracy, 4))
```

```
##  
## Accuracy: 0.7413
```

```
cat("\n Precision:", round(nb_binary_metrics$Precision, 4))
```

```
##  
## Precision: 0.3045
```

```
cat("\n Recall:", round(nb_binary_metrics$Recall, 4))
```

```
##  
## Recall: 0.6675
```

```
cat("\n F1 Score:", round(nb_binary_metrics$F1, 4))
```

```
##  
## F1 Score: 0.4183
```

```
cat("\n\nKNN (K =", best_k_balanced, "):")
```

```
##  
##  
## KNN (K = 15 ):
```

```
cat("\n Accuracy:", round(knn_binary_metrics$Accuracy, 4))
```

```
##  
## Accuracy: 0.7874
```

```
cat("\n Precision:", round(knn_binary_metrics$Precision, 4))
```

```
##  
## Precision: 0.3435
```

```
cat("\n Recall:", round(knn_binary_metrics$Recall, 4))
```

```
##  
## Recall: 0.5771
```

```
cat("\n F1 Score:", round(knn_binary_metrics$F1, 4))
```

```
##  
## F1 Score: 0.4306
```

```
#Multiclass NB vs KNN Comparison
```

```
cat("\n\nMULTICLASS CLASSIFICATION:\n")
```

```
##  
##  
## MULTICLASS CLASSIFICATION:
```

```
cat("\nNaive Bayes:")
```

```
##  
## Naive Bayes:
```

```
cat("\n  Accuracy:", round(nb_multiclass_metrics$Accuracy, 4))
```

```
##  
##   Accuracy: 0.7554
```

```
cat("\n  Macro F1:", round(nb_multiclass_metrics$Macro_F1, 4))
```

```
##  
##   Macro F1: 0.4301
```

```
cat("\n  Class 0 F1:", round(nb_multiclass_metrics$Class0_F1, 4))
```

```
##  
##   Class 0 F1: 0.8521
```

```
cat("\n  Class 1 F1:", round(nb_multiclass_metrics$Class1_F1, 4))
```

```
##  
##   Class 1 F1: 0.032
```

```
cat("\n  Class 2 F1:", round(nb_multiclass_metrics$Class2_F1, 4))
```

```
##  
##   Class 2 F1: 0.4062
```

```
cat("\n\nKNN (K =", best_k_multiclass, "):")
```

```
##  
##  
## KNN (K = 7 ):
```

```
cat("\n  Accuracy:", round(knn_multiclass_metrics$Accuracy, 4))
```

```
##  
##   Accuracy: 0.8348
```

```
cat("\n  Macro F1:", round(knn_multiclass_metrics$Macro_F1, 4))
```

```
##  
##   Macro F1: 0.5962
```

```
cat("\n Class 0 F1:", round(knn_multiclass_metrics$Class0_F1, 4))
```

```
##
## Class 0 F1: 0.9084
```

```
cat("\n Class 1 F1:", round(knn_multiclass_metrics$Class1_F1, 4))
```

```
##
## Class 1 F1: NaN
```

```
cat("\n Class 2 F1:", round(knn_multiclass_metrics$Class2_F1, 4))
```

```
##
## Class 2 F1: 0.284
```

```
#Metric Comparison Table
binary_baseline_acc <- max(table(binary_test_scaled$Diabetes_binary)) / nrow(binary_test_scaled)

comparison_table <- data.frame(
  Dataset = c("Binary", "Binary", "Binary", "Multiclass", "Multiclass", "Multiclass"),
  Model = c("Baseline", "Naive Bayes", paste("KNN (K=", best_k_balanced, ")", sep=""),
            "Baseline", "Naive Bayes", paste("KNN (K=", best_k_multiclass, ")", sep="")),
  Accuracy = c(binary_baseline_acc,
               nb_binary_metrics$Accuracy,
               knn_binary_metrics$Accuracy,
               0.3333, # Random guess for 3 classes,
               nb_multiclass_metrics$Accuracy,
               knn_multiclass_metrics$Accuracy),
  F1_Score = c(0,
               nb_binary_metrics$F1,
               knn_binary_metrics$F1,
               0.3333,
               nb_multiclass_metrics$Macro_F1,
               knn_multiclass_metrics$Macro_F1)
)

print(comparison_table)
```

```
##      Dataset      Model Accuracy F1_Score
## 1      Binary    Baseline 0.8606540 0.0000000
## 2      Binary Naive Bayes 0.7412736 0.4182584
## 3      Binary KNN (K=15) 0.7873544 0.4306296
## 4 Multiclass    Baseline 0.3333000 0.3333000
## 5 Multiclass Naive Bayes 0.7553659 0.4301002
## 6 Multiclass KNN (K=7) 0.8347951 0.5962014
```

```
comparison_long <- data.frame(
  Model = rep(comparison_table$Model[2:3], 2),
  Metric = c(rep("F1 Score", 2), rep("Accuracy", 2)),
  Score = c(comparison_table$F1_Score[2], comparison_table$F1_Score[3],
            comparison_table$Accuracy[2], comparison_table$Accuracy[3])
)
```

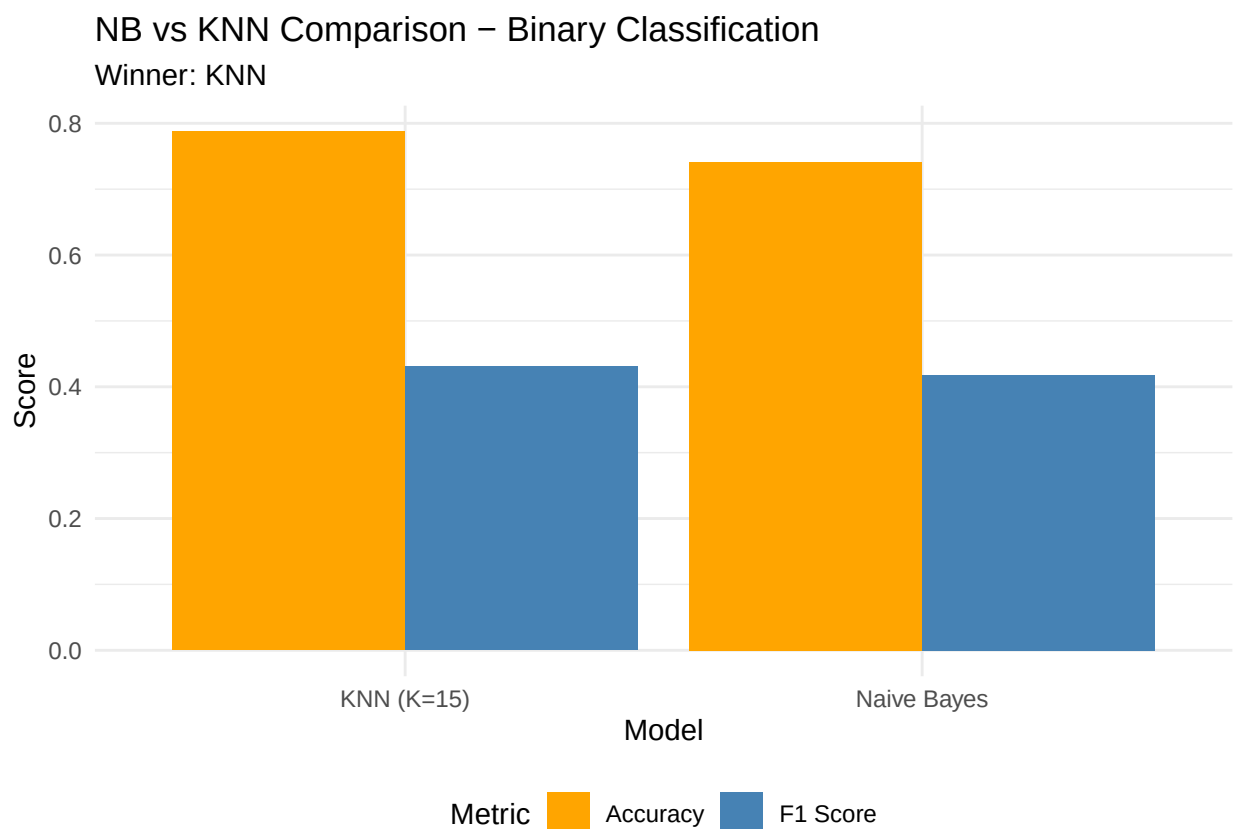
```

        comparison_table$Accuracy[2], comparison_table$Accuracy[3])
    )

comparison_plot <- ggplot(comparison_long, aes(x = Model, y = Score, fill = Metric)) +
  geom_bar(stat = "identity", position = "dodge") +
  labs(title = "NB vs KNN Comparison - Binary Classification",
       subtitle = paste("Winner:", binary_winner),
       y = "Score") +
  theme_minimal() +
  scale_fill_manual(values = c("F1 Score" = "steelblue", "Accuracy" = "orange")) +
  theme(legend.position = "bottom")

print(comparison_plot)

```



```

#Binary NB vs KNN Comparison
cat("\n BINARY CLASSIFICATION:\n")

```

```

##
## BINARY CLASSIFICATION:

```

```

cat("\nNaive Bayes (Balanced):")

```

```

##
## Naive Bayes (Balanced):

```

```
cat("\n Accuracy:", round(nb_binary_metrics$Accuracy, 4))
```

```
##  
## Accuracy: 0.7413
```

```
cat("\n Precision:", round(nb_binary_metrics$Precision, 4))
```

```
##  
## Precision: 0.3045
```

```
cat("\n Recall:", round(nb_binary_metrics$Recall, 4))
```

```
##  
## Recall: 0.6675
```

```
cat("\n F1 Score:", round(nb_binary_metrics$F1, 4))
```

```
##  
## F1 Score: 0.4183
```

```
cat("\n\nKNN (K =", best_k_balanced, "):")
```

```
##  
##  
## KNN (K = 15 ):
```

```
cat("\n Accuracy:", round(knn_binary_metrics$Accuracy, 4))
```

```
##  
## Accuracy: 0.7874
```

```
cat("\n Precision:", round(knn_binary_metrics$Precision, 4))
```

```
##  
## Precision: 0.3435
```

```
cat("\n Recall:", round(knn_binary_metrics$Recall, 4))
```

```
##  
## Recall: 0.5771
```

```
cat("\n F1 Score:", round(knn_binary_metrics$F1, 4))
```

```
##  
## F1 Score: 0.4306
```

```

#Multiclass NB vs KNN Comparison
cat("\n\n MULTICLASS CLASSIFICATION:\n")

##
##
##  MULTICLASS CLASSIFICATION:

cat("\nNaive Bayes:")

##
## Naive Bayes:

cat("\n  Accuracy:", round(nb_multiclass_metrics$Accuracy, 4))

##
##  Accuracy: 0.7554

cat("\n  Macro F1:", round(nb_multiclass_metrics$Macro_F1, 4))

##
##  Macro F1: 0.4301

cat("\n  Class 0 F1:", round(nb_multiclass_metrics$Class0_F1, 4))

##
##  Class 0 F1: 0.8521

cat("\n  Class 1 F1:", round(nb_multiclass_metrics$Class1_F1, 4))

##
##  Class 1 F1: 0.032

cat("\n  Class 2 F1:", round(nb_multiclass_metrics$Class2_F1, 4))

##
##  Class 2 F1: 0.4062

cat("\n\nKNN (K =", best_k_multiclass, "):")

##
##
## KNN (K = 7 ):

cat("\n  Accuracy:", round(knn_multiclass_metrics$Accuracy, 4))

##
##  Accuracy: 0.8348

```



```
cat("\n Macro F1:", round(knn_multiclass_metrics$Macro_F1, 4))
```

```
##
```

```
## Macro F1: 0.5962
```

```
cat("\n Class 0 F1:", round(knn_multiclass_metrics$Class0_F1, 4))
```

```
##
```

```
## Class 0 F1: 0.9084
```

```
cat("\n Class 1 F1:", round(knn_multiclass_metrics$Class1_F1, 4))
```

```
##
```

```
## Class 1 F1: NaN
```

```
cat("\n Class 2 F1:", round(knn_multiclass_metrics$Class2_F1, 4))
```

```
##
```

```
## Class 2 F1: 0.284
```